

PoC: URL Shortener

Ninad Divekar(256)

Objective

Design a basic web-based URL shortener that takes a long URL and returns a shortened version, similar to services like bit.ly or tinyurl.

Tools & Technologies Used

- Python 3
- Flask (Web framework)
- Dictionary (in-memory data storage)
- Random string generator

How It Works

1. User inputs a long URL using a form (via Flask web page).
2. The backend generates a unique short code (6-character).
3. The long URL is stored in memory using a dictionary with the short code as the key.
4. When someone accesses the short URL, they are redirected to the original long URL.

Code Implementation

```
import re  Untitled-1  from flask import Flask, request, redirect, render_template_string  Untitled-2
1  from flask import Flask, request, redirect, render_template_string
2  import string, random
3
4  app = Flask(__name__)
5  url_mapping = {}
6
7  def generate_short_code(length=6):
8      return ''.join(random.choices(string.ascii_letters + string.digits, k=length))
9
10 @app.route('/', methods=['GET', 'POST'])
11 def home():
12     if request.method == 'POST':
13         long_url = request.form['long_url']
14         short_code = generate_short_code()
15         url_mapping[short_code] = long_url
16         return f"Shortened URL: http://localhost:5000/{short\_code}"
17     return '''
18     <form method="POST">
19         Long URL: <input name="long_url">
20         <input type="submit">
21     </form>
22     '''
23
24 @app.route('/<short_code>')
25 def redirect_to_long_url(short_code):
26     long_url = url_mapping.get(short_code)
27     if long_url:
28         return redirect(long_url)
29     return "URL not found", 404
30
31 if __name__ == '__main__':
32     app.run(debug=True)
33
```

Sample Input & Output

Input:

User enters: <https://www.example.com/articles/how-to-learn-python>

Output:

Shortened URL displayed: `http://localhost:5000/Xy3fT9`

Accessing `http://localhost:5000/Xy3fT9` will redirect to the original long URL.

Future Improvements

- Store URLs in a persistent database (e.g., SQLite).
- Add user authentication and URL history.
- Track URL usage statistics (clicks).
- Use base62 encoding of an auto-incrementing ID instead of random codes.